Project Overview: Personal Expense Tracker Web App

Description:
A **Personal Expense Tracker** is a web application that allows users to **record, manage, and analyze their daily financial expenses**. It provides an **easy interface to log income and spending**, categorize them, and view **summaries or visual reports** to better understand spending habits.

Users will be able to:

- Create an account and log in securely
- Add, edit, and delete expense entries
- Categorize expenses (e.g. food, travel, bills, entertainment)
- View expenses in tables and charts
- Get monthly summaries of spending

🎯 Aim of the Project

- To build a **full-stack Python-based web application** that helps users **track and analyze their personal finances.**
- To **apply core web development skills** (Flask, Python, SQL, HTML, CSS, JS) in a real-world project.
- To demonstrate how to:
    - Handle user authentication
    - Perform CRUD operations
    - Build analytics features
    - Deploy a functional web application

In short:

> **⚡ Empower users to gain control over their finances by providing a simple, secure, and visual way to track expenses.**

💼 Use Cases

Here are the practical use cases for this project:

👤 For Individual Users

- **Track daily spending** to understand where their money goes
- **Set monthly budgets** and compare against actual spending
- **Categorize expenses** (e.g. transport, groceries, entertainment)
- **Analyze spending trends** using charts (e.g. monthly totals, category-wise breakdown)

📊 For Students

- Manage pocket money and student allowances
- Monitor how much is being spent on food, travel, and leisure
- Save for specific goals by tracking daily spending patterns

---

🧮 For Freelancers / Small Business Owners

- Track small business-related expenses
- Generate quick monthly summaries for tax or budget planning
- Organize receipts/expenses per category for easier bookkeeping

---

⚙️ Core Features Summary

Feature	Description
User Authentication	Login, Signup, Logout using Flask-Login
Expense CRUD	Add, edit, delete, view expenses
Categories & Filters	Organize and filter expenses by category/date
Analytics & Charts	Visual spending analysis using Chart.js
Responsive UI	Clean Bootstrap-based dashboard
Deployment Ready	Hosted on Render / Railway

📅 2-Week Day-by-Day Plan

🗓️ Week 1 — Core System Development

Goal: Build project skeleton, database, authentication, and expense CRUD.

Day 1: Project Setup & Flask Basics

- Install Python & create virtual environment
- Install Flask, Flask SQLAlchemy, Flask-WTF
- Set up basic Flask project structure (`app.py`, `templates/`, `static/`)
- Create “Hello World” route

Day 2: Database & Models

- Create SQLite database
- Add `User` and `Expense` models
- Set up SQLAlchemy and initialize database
- Add Flask-Migrate for migrations

Day 3: User Authentication (Signup)

- Add user registration form (Flask-WTF)
- Add password hashing (werkzeug.security)
- Save users to database

Day 4: User Authentication (Login/Logout)

- Implement login/logout routes
- Use Flask-Login to handle sessions
- Restrict dashboard to logged-in users

Day 5: Expense CRUD (Create & Read)

- Add form to create a new expense
- Save expenses with foreign key (user_id)
- Show logged-in user's expenses in a table

Day 6: Expense CRUD (Update & Delete)

- Add edit and delete buttons for each expense
- Implement update and delete routes

Day 7: UI Design (Basic Templates)

- Create templates: base.html, login.html, register.html, dashboard.html
- Add Bootstrap styling for forms and table layout

🗓️ Week 2 — Features, Analytics & Deployment

Goal: Enhance features, improve UI, add charts, test, and deploy.

Day 8: Filtering & Sorting

- Add filters: by category, by date
- Sort by amount/date
- Improve table layout with Bootstrap

Day 9: Dashboard Analytics

- Add total expenses calculation
- Calculate monthly totals and category-wise sums
- Create helper functions for analytics

Day 10: Charts Integration

- Add Chart.js to show:
    - Monthly spending trend (line chart)
    - Category distribution (pie chart)

Day 11: User Experience Enhancements

- Add flash messages for success/errors
- Add navigation bar and logout link
- Make design responsive

Day 12: Testing & Error Handling

- Add input validation in forms
- Add try/except error handling in routes
- Write a few unit tests using `unittest` or `pytest`

Day 13: Deployment Prep

- Move secrets to `.env` file
- Create `requirements.txt`
- Set up production-ready `app.py` with `gunicorn`
- Test app locally in production mode

Day 14: Deployment & Final Polish

- Deploy to Render / Railway / PythonAnywhere
- Test all features live
- Fix any final UI bugs

✅ After Completion (Optional Upgrades)

- Export expenses to CSV
- Use PostgreSQL on production
- Add password reset via email
- Add recurring expense tracking

📚 Core Python & Flask Resources

🐍 Python Basics (if you need to brush up)

- Python Tutorial – W3Schools
- Python Full Course for Beginners (YouTube, 4hr)

🌐 Flask Web Framework

- Official Flask Docs
- Flask Crash Course (YouTube – Traversy Media) — Beginner-friendly and practical
- Flask Mega-Tutorial (Miguel Grinberg) — In-depth and covers everything from setup to deployment

---

🗃️ Database and ORM

- Flask SQLAlchemy Docs
- SQLAlchemy Docs
- Flask SQLAlchemy Crash Course (YouTube)
- SQLite Tutorial – W3Schools

Extra:

- Flask-Migrate Docs — For database migrations

🔐 User Authentication

- Flask-Login Docs
- Flask Authentication Tutorial (YouTube) — Very practical and beginner friendly
- Password Hashing with Werkzeug

🎨 Frontend (UI, Styling & Charts)

- Bootstrap Docs — For styling your templates
- Bootstrap Crash Course (YouTube)
- Chart.js Docs — For adding charts to your dashboard
- Chart.js Tutorial (YouTube)

🧪 Testing, Security, and Deployment

- unittest (Python)
- pytest Quickstart
- python-dotenv Docs — For managing environment variables
- Deploy Flask on Render (Guide)
- Deploy Flask on Railway (YouTube)

📁 Project-Based Guides (Very Useful)

- Flask Expense Tracker Project (YouTube) — Beginner-friendly full-stack expense tracker
- Flask CRUD App Example (YouTube) — Shows how to build full CRUD logic
- Flask Project Folder Structure Best Practices

✅ Suggested Learning Order

1. Flask basics + HTML templates
2. Flask SQLAlchemy + database models
3. User auth with Flask-Login
4. CRUD operations for expenses
5. Bootstrap UI integration
6. Chart.js analytics
7. Testing + deployment

🖥️ Frontend

(React + Tailwind CSS + Chart.js)

⚛️ React

- 📖 Official React Docs – Start here, very beginner friendly
- 📹 React Crash Course (Traversy Media) – Great overview in 1.5 hr
- 📦 Vite + React Setup Guide – To scaffold your React project quickly

🎨 Tailwind CSS

- 📖 Tailwind Docs – Best place to learn syntax
- 📹 Tailwind CSS Crash Course (YouTube) – Beginner-friendly
- 💡 Tailwind UI Components – Copy-paste ready UI blocks

📊 Chart.js

- 📖 Chart.js Docs – Covers all chart types and options
- 📹 Chart.js + React Tutorial – Shows how to add charts in React dashboard

---

⚙️ Backend

(FastAPI/Flask + Python)

⚡ FastAPI

- 📖 FastAPI Official Docs – Extremely well-written
- 📹 FastAPI Crash Course (freeCodeCamp) – Full project-based tutorial (4 hr)

- 📁 FastAPI Project Structure Guide – How to structure production projects

🧪 Pydantic (used in FastAPI for data validation)

- 📖 Pydantic Docs
- 📽️ Pydantic Tutorial (YouTube)

🗃️ Database & ORM

(PostgreSQL + SQLAlchemy + Alembic)

🐘 PostgreSQL

- 📖 PostgreSQL Tutorial (W3Schools)
- 📽️ PostgreSQL Crash Course (YouTube) – Basics in 1 hr

🧩 SQLAlchemy ORM

- 📖 SQLAlchemy Docs
- 📽️ SQLAlchemy with FastAPI Tutorial – Great starter project

🧩 Alembic (for migrations)

- 📖 Alembic Docs
- 📽️ Alembic Migrations with FastAPI (YouTube)

🔐 Authentication

(JWT Auth with FastAPI)

- 📖 FastAPI Security Docs (JWT Auth)
- 📽️ FastAPI JWT Auth Tutorial (YouTube) – Step-by-step token-based login/register
- 📦 python-jose – Library for creating JWTs
- 📦 passlib – For password hashing

☁️ Deployment

(Frontend: Vercel | Backend: Render + Docker)

🌐 Frontend Deployment (Vercel)

- 📖 Deploy React App to Vercel Docs
- 📽️ React + Vercel Deployment Tutorial

⚙️ Backend Deployment (Render)

- 📖 Deploy FastAPI on Render Docs
- 📽️ Deploy FastAPI to Render (YouTube)

📦 Docker (for backend)

- 📖 Docker Getting Started Guide
- 📽️ Docker Crash Course (YouTube) – Easy and beginner friendly
- 📖 Dockerizing a FastAPI App

🧪 Bonus (Optional But Industry-Relevant)

- **GitHub Actions (CI/CD)** → Guide
- **.env handling** → python-dotenv Docs
- **Testing with Pytest** → Pytest Docs

📌 Suggested Learning Path

Follow this order during your 2-week timeline:

1. **Week 1 (Days 1–4):** FastAPI basics + SQLAlchemy + DB models
2. **Week 1 (Days 5–6):** JWT Auth + CRUD APIs (expenses, users)
3. **Week 2 (Days 1–3):** React + Tailwind frontend, connect APIs
4. **Week 2 (Days 4–5):** Add Chart.js dashboard + polish UI
5. **Week 2 (Day 6–7):** Dockerize backend + deploy on Render and Vercel